Energy Storms Parallel Implementation Report

Andrea D'Aniello (2122222) and Daniele Forgione (1668469)

This report presents the implementation choices, the development process, and the final performance evaluation.

---

1 Implemented solution

1.1 MPI and OpenMP version

The layer is divided into contiguous regions, **one for each MPI process**. When the size is not divisible by the number of processes, the first ranks receive **one extra cell**. Every rank stores its local region and two ghost cells. This supports the three-point relaxation stencil without replicating the complete layer.

The impact phase is organized in blocks of 1,024 cells. OpenMP distributes the blocks with static scheduling, and the inner cell loop uses **omp simd**. Each thread updates different cells, so no atomic operation is required. We also precompute sqrt(distance + 1) in an attenuation table. This removes the **same expensive square-root calculation from the particle-cell loop**, at the cost of extra memory.

Before relaxation, neighbouring ranks exchange boundary values using MPI_Irecv and MPI_Isend. After MPI_Waitall, every rank computes its local relaxed values. A second halo exchange provides the neighbours needed for peak detection. Each OpenMP thread finds a local candidate; a short critical section combines the candidates, and MPI_Allreduce with MPI_MAXLOC selects the global result while preserving the smallest-position tie rule.

1.2 CUDA version

The CUDA version uses **blocks of 256 threads**. In the impact kernel, one thread is assigned to one layer cell and processes the particles of the current storm in their original order. **A separate kernel computes relaxation** after a device-to-device copy. Two layer arrays stay allocated on the GPU, while the current storm is **transferred before its kernels run**. The relaxed layer is then copied to the CPU, where the maximum is found **serially**.

2 Development process and alternatives

Our first OpenMP approach created a parallel loop inside the particle loop. It repeatedly created and synchronized a thread team, so **the overhead was too large**. Moving the parallelism to the cells reduced this overhead, **but the loop structure also affected vectorization**. The final blocked organization was chosen to **keep a coarse OpenMP workload** while still giving the compiler an explicit SIMD.

```
int block_size = local_size / omp_get_max_threads();
    if (block_size < 1) block_size = 1;
    if (block_size > CELL_BLOCK) block_size = CELL_BLOCK;

    for( i = 0; i < num_storms; i++ ) {

        /* 4.1. Ogni thread lavora su blocchi piccoli, che restano in cache.
           Le particelle restano in ordine; SIMD lavora su celle indipendenti. */
        #pragma omp parallel for schedule(static)
        for (int first = 1; first <= local_size; first += block_size) {
            int end = first + block_size;
            if (end > local_size + 1) end = local_size + 1;
            for (int j = 0; j < storms[i].size; j++) {
                float energy = ((float)storms[i].posval[j*2+1] * 1000) / layer_size;
                int position = storms[i].posval[j*2];
                #pragma omp simd
                for (int cell = first; cell < end; cell++) {
                    int k_glob = start_index + cell - 1;
                    update(local_layer, attenuation, threshold, cell, k_glob,   position,
energy);
                }
            }
        }
```

A second important change was the attenuation table. The original sequential reference evaluates sqrtf **for every particle-cell pair.** The optimized 1-process, 1-thread version is already about 11.5 times faster than the approximately 458 s sequential baseline. For this reason, the scaling calculations later in the report use the optimized **1 x 1 version as their reference**.

We initially used blocking MPI exchanges and also experimented with MPI activity close to OpenMP regions. The final version keeps MPI calls **outside the threaded regions** and uses non-blocking **point-to-point operations**.

For CUDA, we kept the maximum search **on the CPU**. A device reduction would reduce transfers and serial work, but it would also require careful handling of local peaks and equal-value positions. We chose the **simpler implementation first** and verified that it produces the expected results on the measured workloads.

3 Experimental method

The main strong-scaling workload is test_07: **a layer of 1,000,000 cells and four storms with 5,000 particles each**. The workload is fixed while P and T change. The weak-scaling tests keep **two OpenMP threads per process and 500,000 cells per worker**: the layer grows from 1 to 8 million cells as the process count grows from 1 to 8. The number of storms and particles per storm stays fixed.

Each strong-scaling configuration was executed three times. The weak-scaling configurations were also repeated three times. CUDA test_07 was run four times. We report the arithmetic mean and sample standard deviation. Strong-scaling speedup is $S(q) = T(1) / T(q)$, and efficiency is $E(q) = S(q) / q$. Weak-scaling efficiency is $T(1) / T(q)$.

The sequential reference is one approximate measurement of 458 s (in cluster), so it is used only for an overall comparison. It is not used to estimate variability or parallel efficiency. File loading and output are outside the measured core time. MPI timings include **computation, halo communication, allocation and cleanup inside the core**; CUDA timings include **device allocation, transfers, kernels, synchronization, the host maximum search and cleanup.**

4 Experimental evaluation

4.1 Correctness

All repeated runs of test_07 produced the **same printed positions and values as the reference**. We also compared the sequential and hybrid versions on small tests that exercise different peak positions and on test_09 with both 16 and 17 cells. The printed results **matched in every case.**

Case	Layer	Reference and MPI result
test_03	20	10 12966.631836
test_04	20	16 12966.632812
test_05	20	9 12829.288086
test_06	20	16 12829.288086
test_09	16	5 189.004868
test_09	17	15 193.299606

Table 1. Correctness checks; values are shown at the precision printed by the programs.

4.2 Strong scaling

P	T	Workers	Runs	Mean s	SD s	Speedup	Efficiency
1	1	1	3	39.942	0.164	1.00	100.0%
2	2	4	3	11.058	0.145	3.61	90.3%
4	2	8	3	6.035	0.166	6.62	82.7%
8	2	16	3	3.499	0.281	11.41	71.3%
16	2	32	3	2.264	0.232	17.64	55.1%

Table 2. Strong scaling on test_07. Speedup and efficiency use the optimized 1 x 1 mean as reference.

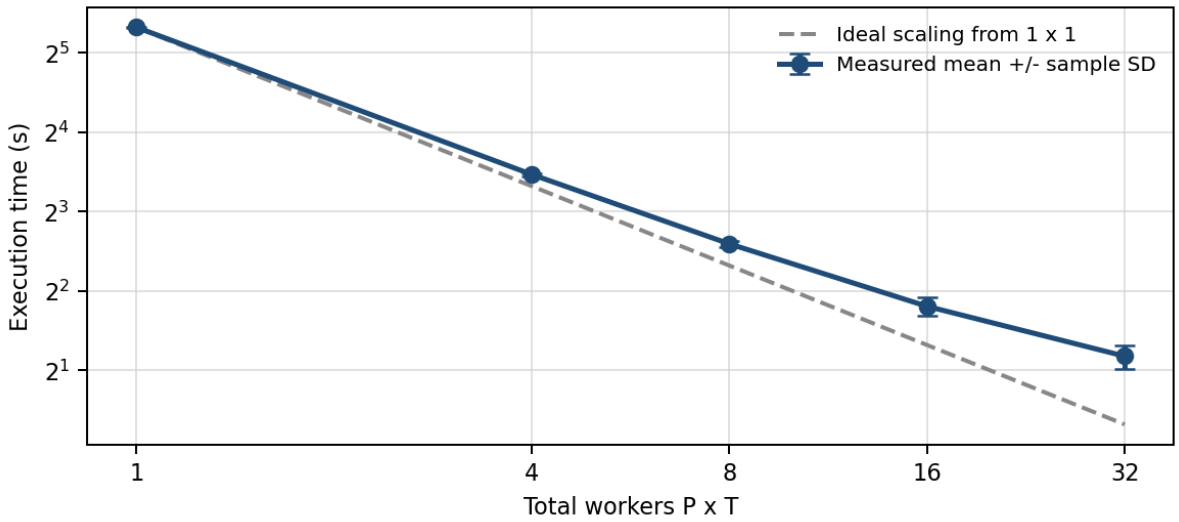


Figure 1. Strong-scaling time. Error bars show the sample standard deviation.

The 16 x 2 configuration reaches a speedup of 17.64 with 32 workers and an efficiency of 55.1%. Scaling is good up to 8 workers, **then efficiency decreases as each rank receives less computation while synchronization**, halo exchange and memory costs become more important. Even so, the final hybrid mean is 2.264 s, **about 202 times faster than the original sequential reference** when algorithmic and parallel improvements are considered together.

4.3 Weak scaling

P	T	Layer cells	Runs	Mean s	SD s	Weak efficiency
1	2	1 M	3	20.536	0.034	100.0%
2	2	2 M	3	21.588	0.105	95.1%
4	2	4 M	3	24.229	0.093	84.8%
8	2	8 M	3	29.190	0.442	70.4%

Table 3. Weak scaling with two threads per process and 500,000 cells per worker.

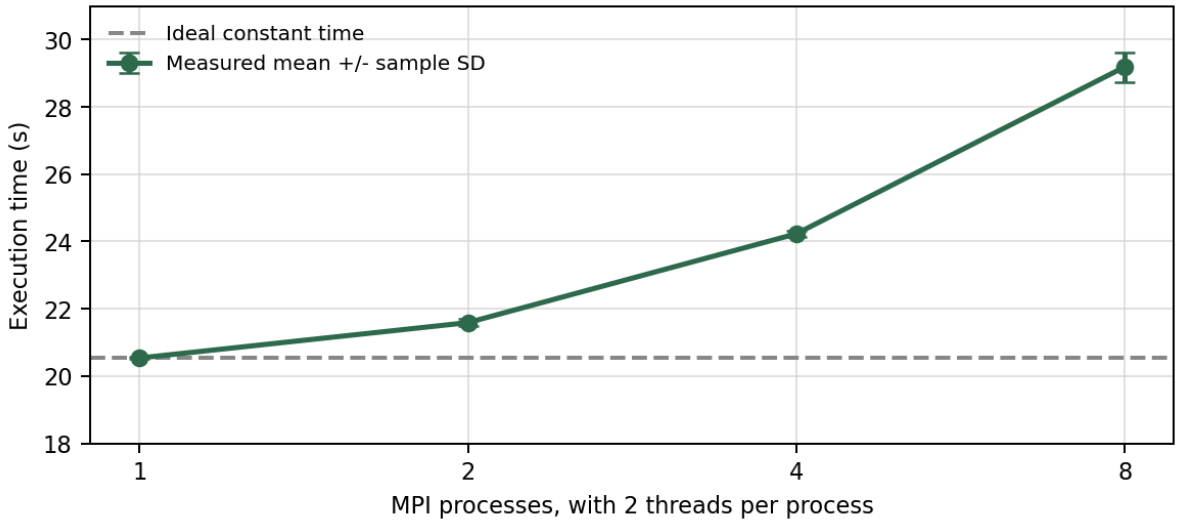


Figure 2. Weak-scaling time. Ideal behavior would keep the execution time constant.

The mean time grows from 20.536 s to 29.190 s, giving 70.4% weak-scaling efficiency at 8 processes and 16 threads. **The trend is gradual**. It is consistent with increasing communication, synchronization and per-process initialization costs.

4.4 CUDA and overall comparison

Version	Resources	Runs	Mean time	Sample SD	Speedup vs sequential
Sequential reference	1 CPU core	1	about 458 s	-	1.0
Optimized hybrid	16 MPI x 2 OMP	3	2.264 s	0.232 s	202.3
CUDA	1 GPU	4	0.381 s	0.081 s	1202

Table 4. End-to-end comparison on test_07. Sequential speedups include algorithmic changes, so they are not pure scaling results.

The CUDA sample standard deviation is 0.081 s, smaller in absolute terms than the 0.232 s of the best hybrid configuration. However, because the mean times are very different, we also compare the coefficient of variation:

$$CV_{cuda} = \frac{0.081}{0.381} \times 100 \approx 21\%$$

$$CV_{hybrid} = \frac{0.232}{2.264} \times 100 \approx 10\%$$

Therefore, CUDA has **lower absolute variability but greater relative variability**. All four CUDA runs returned the same printed result.

5 Limitations and possible improvements

Memory use: The attenuation table is replicated by every MPI process and **grows with the maximum distance**. For very large layers this can consume **substantial memory**. A shared table inside each node, a smaller representation, or computing selected values on demand could reduce this cost.

Communication: Non-blocking MPI is used, but the immediate wait **prevents overlap**. Internal cells could be relaxed while halo messages are in flight, followed by the boundary cells after completion.

CUDA data movement. After each storm, the complete layer is copied from the GPU to the CPU to find the maximum. This transfer **may become expensive** for large layers, even though only the maximum value and its position are required. A possible improvement is to perform a **parallel reduction on the GPU and transfer only the final result.**

Portability and robustness: The chosen block sizes and cache block length **may not be optimal on every CPU or GPU**. They should be tuned on other systems.

6 Conclusions

The experiments show that the most suitable implementation **depends on the available hardware**. The hybrid version provides a portable solution and scales effectively across CPU resources, while CUDA is preferable for this computation when a GPU is available.

The main lesson from the development process is that **adding parallelism is not sufficient by itself**: thread-management overhead, memory access patterns and data transfers must also be considered.